

Algoritmos de Rede Neural Artificial aplicados ao dataset MNIST: implementação e exploração

Luiz Eduardo Foschiera Couto Lopes
255383

Henrique De Sordi
251143

Tallyta Duarte
188305

Resumo—Algoritmos de aprendizado de máquina e de aprendizado profundo têm se tornado um dos pilares da evolução tecnológica e científica das últimas décadas. Em especial, algoritmos de rede neural se destacam como versáteis e adequados para o cenário do Big Data. Uma das aplicações de estudo de Redes Neurais é a identificação de dígitos manuscritos e, para a exploração dessa tarefa, são utilizados datasets como o MNIST. O dataset MNIST consiste em um conjunto de imagens em escala de cinza de dígitos manuscritos, rotuladas em dez classes. Neste trabalho foi utilizado um subset de 5000 imagens dele, de forma vetorizada em que cada imagem de 20×20 pixels foi representada por um vetor de 400 atributos. A aplicação do uso de redes neurais e tratamento de imagens envolve habilidades aplicáveis em diversas áreas, como reconhecimento de caracteres, leitura automática de documentos e identificação de padrões visuais. Nesse sentido, o objetivo deste trabalho é explorar a implementação manual e parâmetros de treinamento de redes neurais para o problema de identificação de dígitos manuscritos. Foram desenvolvidos experimentos como análise de curvas de aprendizado em função do tamanho do conjunto de treinamento, a investigação do efeito da constante de regularização λ , a comparação entre o gradiente descendente e o gradiente conjugado, a análise qualitativa dos erros de classificação e a visualização das representações aprendidas na camada oculta. A melhor configuração alcançou 93,1% de acurácia no conjunto de teste com gradiente descendente e 93,8% com gradiente conjugado. Dessa forma, buscou-se fundamentar na prática os conceitos estudados na disciplina de Aprendizado de Máquina, tornando claras etapas que geralmente são abstraídas por bibliotecas modernas.

Index Terms—Reconhecimento de dígitos manuscritos, Redes Neurais Artificiais, MNIST

I. INTRODUÇÃO

A. Cenário Geral

Redes Neurais Artificiais (RNAs) são modelos computacionais compostos por unidades interligadas que transformam uma entrada por meio de parâmetros ajustáveis. No aprendizado supervisionado, esses parâmetros são estimados a partir de exemplos rotulados, de modo que a rede associe padrões às classes de interesse. Apesar da inspiração biológica, uma RNA é um modelo matemático: seu comportamento é definido pela arquitetura, pelas funções de ativação, pela função de custo e pelo procedimento de otimização [1].

Os fundamentos dessa abordagem remontam ao neurônio formal de McCulloch e Pitts, que descreveu a primeira concepção de neurônio lógico e computacional [2]. Posteriormente, o perceptron de Rosenblatt introduziu um modelo capaz de ajustar seus parâmetros para tarefas de classificação [3]. Um marco para redes com camadas ocultas foi o uso da

retropropagação do erro, que permite calcular gradientes dos pesos e dos vieses e reduzir de forma iterativa a distância entre as saídas produzidas e as desejadas, medida por uma função de perda [4].

O desenvolvimento de arquiteturas com múltiplas camadas, a disponibilidade de bases de dados maiores e o aumento da capacidade computacional tornaram as redes neurais uma das abordagens mais importantes do aprendizado de máquina moderno. Em particular, modelos profundos aprendem representações em sucessivos níveis de abstração e obtiveram avanços expressivos em reconhecimento de fala, visão computacional e outros domínios [5]. Essas técnicas integram o campo mais amplo da Inteligência Artificial (IA), que também inclui, por exemplo, métodos simbólicos, busca e otimização.

B. Motivação

Bibliotecas atuais permitem construir e treinar redes neurais com poucas linhas de código. Essa conveniência é positiva em aplicações práticas e de indústria, mas pode ocultar os mecanismos que ditam a forma como o modelo funciona. A implementação manual adotada neste trabalho torna explícitos a propagação direta, a codificação das classes, a função de custo regularizada, o cálculo dos gradientes e a atualização dos pesos, ou seja, as diversas etapas necessárias para o entendimento desse tipo de algoritmo. Ela também permite comparar o gradiente descendente com gradientes conjugados, investigar o efeito do parâmetro de regularização λ e observar a evolução dos pesos dos neurônios ocultos.

A tarefa escolhida é a classificação de dígitos manuscritos. O MNIST (*Modified National Institute of Standards and Technology*) é a referência clássica para esse problema: sua versão canônica possui 60.000 imagens de treino e 10.000 de teste, cada uma representada em escala de cinza com 28×28 pixels e pertencente a uma das dez classes de dígitos [6]. A base se tornou um referencial por combinar uma tarefa visual intuitiva, rótulos disponíveis, dimensão suficiente para avaliar generalização e custo computacional compatível com experimentos menores.

O conjunto efetivamente empregado neste projeto corresponde a uma versão reduzida dessa mesma tarefa: os arquivos de dados contêm 5.000 exemplos, cada qual vetorizado em 400 atributos, equivalentes a imagens de 20×20 pixels, e rótulos codificados de 1 a 10. Dessa forma, o problema é desafiador o bastante para estudar otimização e regularização,

mas permanece compatível com o objetivo de implementar e analisar cada etapa do algoritmo.

C. Fundamentação Teórica

Uma rede neural é composta por unidades computacionais conectadas entre si chamadas de neurônios, os quais são organizados em camadas. Seu funcionamento se dá pelo recebimento de um vetor de atributos $x = [1 \ x_1 \ x_2 \ \dots \ x_n]$, onde o elemento 1 é o chamado *bias*, que é submetido às operações matemáticas de cada camada da rede neural, fornecendo ao final um vetor de saída cujo significado depende da aplicação da rede em questão [1].

As operações matemáticas que caracterizam a rede se organizam de forma que o neurônio i da camada j receba um valor $z_i^{(j)}$, onde

$$z_i^{(j)} = \sum_k \theta_{ik}^{(j-1)} a_k^{(j-1)} \quad (1)$$

com $a_k^{(j-1)}$ sendo a saída da unidade k da camada anterior e $\theta_{ik}^{(j-1)}$ sendo o peso que conecta a unidade em questão ao neurônio. $z_i^{(j)}$ então é processada pela função de ativação $h_\theta(x) = g(z_i^{(j)})$, fornecendo uma das entradas para a próxima camada ou uma das componentes do vetor de saída caso esta seja a última camada de neurônios. Para o caso da primeira camada, o valor $a_k^{(j-1)}$ é a k -ésima entrada do vetor x . O neurônio com $i = 0$ de cada camada será sempre o *bias*, com $a_0^{(j)} = 1$.

Diferentes funções de ativação podem ser escolhidas para compor uma rede neural. Neste trabalho foi utilizada a função sigmoide, uma das ativações clássicas empregadas em redes neurais [1]:

$$g(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

O valor dos pesos θ_{ik}^j , por sua vez, é um dos elementos mais fundamentais a ser determinado para o funcionamento da rede. Sua inicialização pode ser feita arbitrariamente ou de forma aleatória, mas para obter uma rede neural funcional é necessário encontrar os parâmetros que minimizem sua função de custo $J(\theta)$, o que é feito através do chamado treinamento.

No caso da classificação multiclasse, a função de custo utilizada é a soma das entropias cruzadas binárias para cada classe:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[y_k^{(i)} \log(a_k^{(L,i)}) + (1 - y_k^{(i)}) \log(1 - a_k^{(L,i)}) \right]. \quad (3)$$

O gradiente descendente é um dos métodos fundamentais de minimização utilizados no treinamento de redes neurais [1], e sua expressão matemática é dada por:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} \quad (4)$$

Ou, de forma vetorizada:

$$\Theta^{(l)} \leftarrow \Theta^{(l)} - \alpha \nabla_{\Theta^{(l)}} J$$

Combinado ao gradiente descendente, outra técnica amplamente utilizada que foi empregada é o *backpropagation*, utilizada para obter o valor de $\frac{\partial J}{\partial \theta_j}$ de forma retroativa [1], [4]. Nesse sentido, o fluxo, de forma vetorizada, acontece da seguinte forma: inicializa-se aleatoriamente os valores de $\theta_{ik}^{(j)}$, em seguida, para cada exemplo $(x^{(i)}, y)$ executa-se o *forwardpropagation* a fim de obter valores para as ativações de cada neurônio e a predição final $h_\theta(x^{(i)})$. Então, percorrendo de trás para frente, aplica-se o algoritmo de *backpropagation*, que pode ser resumido, no caso da rede implementada neste trabalho, em:

Última camada $l = L$:

$$\delta^L = a^L - y \quad (5)$$

Para as camadas anteriores $l = L - 1, \dots, 1$, de forma retroativa:

$$\delta^l = (\Theta^l)^T \delta^{l+1} * a^l * (1 - a^l) \quad (6)$$

em que $*$ representa a multiplicação entrada a entrada dos elementos.

Com os elementos δ^l calculados, agora faz-se:

$$\nabla_{\Theta^{(l)}} J = \delta^{(l+1)} (a^{(l)})^T$$

E por fim podemos fazer a atualização com gradiente descendente:

$$\Theta^{(l)} \leftarrow \Theta^{(l)} - \alpha \nabla_{\Theta^{(l)}} J$$

Caso seja optado utilizar regularização, adiciona-se um termo de penalização à função de custo. A regularização por norma L_2 , também relacionada ao chamado *weight decay*, é uma estratégia clássica para restringir a magnitude dos pesos e favorecer generalização [1], [7]. Para $m = 1$, se considerado somente um exemplo:

$$J(\Theta) = \dots + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \left\| \Theta_{:,l}^{(l)} \right\|_F^2 \quad (7)$$

E a atualização com a descida de gradiente recebe o termo:

$$\Theta^l = \dots + \frac{\lambda}{m} \Theta^l \quad (8)$$

D. Aplicações e Impacto

A classificação de imagens constitui uma das aplicações clássicas de redes neurais artificiais, estando associada a problemas como reconhecimento de caracteres, leitura automática de documentos e identificação de padrões visuais. O reconhecimento de dígitos manuscritos foi, inclusive, um dos problemas centrais no desenvolvimento e avaliação de redes convolucionais [8]. Nesse contexto, a classificação dos dígitos manuscritos do MNIST representa um problema simplificado,

porém suficientemente complexo para permitir o estudo dos principais elementos envolvidos no treinamento de uma rede neural.

Mais do que obter elevada acurácia sobre o conjunto de dados, a implementação desenvolvida neste trabalho permite analisar diretamente o comportamento de etapas que normalmente são abstraídas por bibliotecas modernas, como a propagação das ativações, o cálculo dos gradientes por *backpropagation*, a atualização dos pesos e o efeito da regularização. Dessa forma, o problema serve como experimentos diversos para compreender como escolhas de arquitetura e hiperparâmetros influenciam a capacidade de aprendizado e de generalização de um modelo de rede neural.

Além disso, os conceitos investigados não são restritos ao dataset MNIST. Os mesmos princípios matemáticos de otimização, regularização e propagação de gradientes formam a base para redes neurais aplicadas a problemas de maior escala em visão computacional e outras áreas do aprendizado de máquina, como o dataset CIFAR-10 [9]. O uso de redes convolucionais em tarefas de visão de maior escala também foi decisivo para avanços posteriores em reconhecimento de imagens [10].

II. METODOLOGIA EXPERIMENTAL

A. Ambiente Computacional

O projeto foi inteiramente desenvolvido na linguagem *Python*, utilizando o ambiente do *VSCode* e o gerenciador *Git*. No desenvolvimento dos códigos foram empregadas também as bibliotecas *Pandas*, *Matplotlib*, *NumPy*, *Tqdm* e *SciPy*, além das bibliotecas nativas do próprio *Python*.

Em adição às bibliotecas, um código fornecido pelo professor com funções úteis para a implementação das redes neurais foi utilizado. O código contém funcionalidades como a função de ativação, função de custo, gradiente descendente, inicialização de parâmetros e implementação dos testes da rede neural, poupando o esforço de programá-las do zero.

B. Implementação

Como citado anteriormente, a base de dados utilizada para o projeto foi a MNIST, contendo 5000 imagens de dígitos manuscritos na forma de matrizes 20×20 . Os dados estão distribuídos em 2 arquivos, um com as imagens já transformadas em vetores de tamanho 400, para viabilizar sua utilização na rede neural, e outro com os dígitos que identificam cada uma das imagens, correspondentes aos rótulos.

Foi constatado que alguns dados do arquivo de imagens estavam em formato inadequado para processamento pela rede, tanto *floats* em formato de *strings* quanto vírgulas no lugar de ponto para representar as casas decimais, o que foi corrigido criando um código de pré-processamento dos dados, que gera um novo arquivo de dados ajustados.

A rede neural foi então implementada utilizando como função de ativação a sigmoide e como método de minimização da função de custo o gradiente descendente com regularização, além da técnica de *backpropagation* para treinamento da rede.

A arquitetura da rede foi definida utilizando-se 1 camada oculta de 25 neurônios.

C. Escolha de Modelos e Organização dos Dados

O conjunto de dados, por sua vez, foi embaralhado aleatoriamente e dividido em 3 conjuntos: treinamento, teste e validação, seguindo a proporção de 76, 20 e 4% dos dados. Como de costume no *backpropagation*, os pesos foram inicializados aleatoriamente, usando uma função que obedece à distribuição normal, e a ativação de cada neurônio foi inicialmente definida com uma iteração de *forwardpropagation*.

Na etapa da escolha de modelos, diferentes valores para a constante de regularização foram escolhidos e a rede neural foi treinada com cada um deles. Cada rede treinada com um parâmetro de regularização diferente foi testada utilizando o conjunto de dados de validação e o modelo com menor erro de validação foi escolhido para dar prosseguimento a seu treinamento e posterior teste das redes com os dados do conjunto de testes.

D. Experimentos Realizados

Neste trabalho, inicialmente foi construída a rede neural e implementadas todas as suas funcionalidades, como comentado no tópico anterior. Com a rede implementada corretamente, foi conduzida então uma análise de erro obtendo a porcentagem de dígitos previstos erroneamente, juntamente com as imagens destes dígitos e seus rótulos. Similaridade de cossenos entre diferentes classes também foi computada para compreender melhor tipos de erros que podem vir a ser comuns na rede. Em sequência, o algoritmo de checagem do gradiente foi implementado sob uma rede neural muito menor para verificação do cálculo do gradiente.

O mesmo modelo foi agora testado implementando o método do gradiente conjugado e variando o número de iterações e parâmetro de regularização. A evolução dos pesos em cada neurônio também foi acompanhada a cada iteração, gerando uma figura a partir de uma matriz 20×20 normalizada correspondente à i -ésima linha de $\theta^{(1)}$.

Por fim, a seleção de modelos foi executada, calculando os erros de treinamento, teste e validação e automatizando a escolha de λ . Curvas de aprendizado e de erro em função de λ foram plotadas a partir dos resultados.

III. RESULTADOS E DISCUSSÃO

Os experimentos avaliaram a convergência do treinamento, o efeito da quantidade de dados, a influência da regularização, a comparação entre métodos de otimização e a natureza dos erros de classificação. A arquitetura analisada possui 400 atributos de entrada, uma camada oculta com 25 neurônios e 10 classes de saída. Para a seleção de hiperparâmetros e a comparação entre otimizadores, foram utilizados 3800 exemplos de treinamento, 200 de validação e 1000 de teste.

A Tabela I resume o desempenho das melhores configurações selecionadas exclusivamente pela acurácia de validação. O conjunto de teste foi consultado apenas após a seleção de cada configuração.

Tabela I: Desempenho das configurações selecionadas pela validação.

Método	λ	Iterações	Validação (%)	Teste (%)
Gradiente descendente	0	800	92,0	93,1
Gradiente conjugado	3	400	95,0	93,8

A. Convergência e tamanho do conjunto de treinamento

Inicialmente, foi analisada a evolução da função de custo durante o treinamento para diferentes quantidades de exemplos, mantendo-se $\lambda = 0$. A Figura 1 apresenta as curvas de custo para conjuntos entre 100 e 4000 exemplos.

Em todos os tamanhos de conjunto, a função de custo apresenta queda acentuada nas primeiras iterações, seguida de redução progressivamente mais lenta. Esse comportamento indica que o processo de otimização rapidamente encontra regiões de menor custo e, posteriormente, realiza ajustes mais finos nos parâmetros.

Embora todas as curvas apresentem convergência estável, conjuntos muito pequenos atingem custo de treinamento mais baixo. Esse resultado é esperado: com menos exemplos, a rede possui maior capacidade de memorizar os padrões observados. Entretanto, custo baixo no treinamento não implica necessariamente melhor generalização.

A Figura 2 apresenta os erros de classificação em treinamento e validação para seis tamanhos de conjunto. Nesta análise, foi adotado um conjunto fixo de validação com 1000 exemplos; portanto, seus valores são usados para estudar a tendência da generalização e não devem ser comparados diretamente aos valores da Tabela I, obtidos com outra partição.

Com 100 exemplos de treinamento, o erro de treinamento é aproximadamente nulo, enquanto o erro de validação é de cerca de 19,4%. Esse afastamento caracteriza sobreajuste: a rede ajusta-se quase perfeitamente ao subconjunto disponível, mas apresenta baixa capacidade de generalização.

Com o aumento do número de exemplos, o erro de validação diminui para aproximadamente 10,3% com 880 exemplos e atinge o menor valor observado, 6,7%, com 3220 exemplos. Com 4000 exemplos, o erro de validação permanece próximo desse valor, em torno de 7,0%, enquanto o erro de treinamento chega a aproximadamente 5,5%. Assim, a diferença entre treinamento e validação diminui substancialmente quando mais exemplos são utilizados.

A pequena elevação do erro de validação entre 3220 e 4000 exemplos não deve ser interpretada como deterioração sistemática do modelo, pois foi utilizada apenas uma partição fixa de validação. Ainda assim, a tendência global evidencia que o aumento da quantidade de dados reduz o sobreajuste e melhora a generalização, em concordância com a relação entre capacidade, dados e generalização discutida em [1].

B. Influência da regularização

A influência da regularização foi avaliada por meio da entropia cruzada nos conjuntos de treinamento e validação. Os resultados são apresentados na Figura 3.

Para valores de λ entre zero e 0,1, a entropia cruzada de treinamento permanece próxima de 0,385 e a de validação

próxima de 0,536. Portanto, nessa faixa, a regularização não alterou substancialmente o ajuste do modelo. A partir de $\lambda = 0,3$, observa-se aumento gradual do custo; para $\lambda = 10$, a entropia cruzada de validação chega a aproximadamente 0,75.

A mesma tendência aparece no erro de classificação de validação, mostrado na Figura 4.

Para λ entre zero e 0,1, o erro de validação permaneceu em 8,0%. Para $\lambda = 0,3$, 1 e 3, o erro aumentou para 8,5%; com $\lambda = 10$, atingiu 10,0%. Como o conjunto de validação possui 200 exemplos, a resolução da acurácia é de 0,5 ponto percentual. Logo, diferenças inferiores a essa escala não devem ser superinterpretadas.

A melhor configuração segundo a validação foi $\lambda = 0$, com acurácia de 92,0% e acurácia de teste de 93,1%. Portanto, nesta arquitetura e neste conjunto de dados, não foi observada melhora mensurável de generalização com regularização positiva. Regularização excessiva reduziu a capacidade de ajuste da rede, resultado compatível com o compromisso entre ajuste aos dados e controle da magnitude dos pesos promovido pelo *weight decay* [1], [7].

C. Comparação dos métodos de otimização

A Figura 5, apresentada no Apêndice, compara a acurácia de validação do gradiente descendente e do gradiente conjugado para diferentes números de iterações e valores de λ .

O gradiente descendente apresentou crescimento progressivo da acurácia conforme aumentou o número de iterações. Com 50 iterações, as acurácias de validação variaram entre 79,0% e 79,5%. Com 200 iterações, atingiram aproximadamente 90,0%, e o melhor resultado foi 92,0%, obtido com 800 iterações e $\lambda = 0$. Vale ressaltar que muitas das curvas ficaram sobrepostas, mas incluídas, por questão de completude.

O gradiente conjugado alcançou resultados superiores com menos iterações. Já com 50 iterações, suas acurácias variaram entre 90,5% e 93,0%. O melhor resultado de validação foi 95,0%, obtido com 400 iterações e $\lambda = 3$. Essa configuração alcançou 93,8% no conjunto de teste, enquanto a melhor configuração de gradiente descendente alcançou 93,1%.

Assim, o gradiente conjugado apresentou vantagem clara em velocidade de convergência no conjunto de validação. No entanto, a diferença no teste foi de apenas 0,7 ponto percentual. Como os resultados foram obtidos com uma única partição e uma única semente, essa diferença deve ser interpretada como evidência indicativa, e não como demonstração estatística definitiva de superioridade. Uma avaliação mais robusta exigiria repetições com diferentes sementes ou validação cruzada.

D. Análise das classificações incorretas

A Figura 6, apresentada no Apêndice, mostra os primeiros 25 erros da configuração baseada em gradiente descendente com $\lambda = 0$. Essa configuração obteve acurácia de teste de 93,1%; portanto, dos 1000 exemplos de teste, 69 foram classificados incorretamente.

Os erros exibidos incluem confusões como $3 \rightarrow 2$, $7 \rightarrow 9$, $9 \rightarrow 4$, $5 \rightarrow 2$ e $6 \rightarrow 2$. Em vários casos, os dígitos possuem traços inclinados, laços incompletos ou formatos

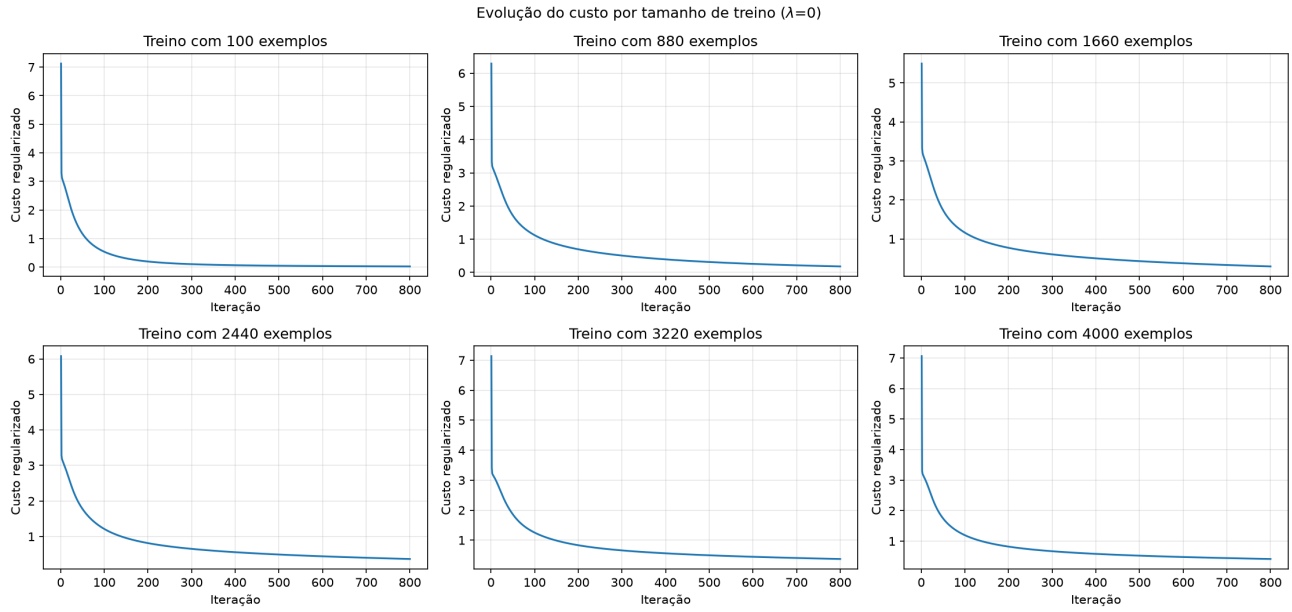


Figura 1: Evolução do custo de treinamento para diferentes tamanhos de conjunto, com $\lambda = 0$.

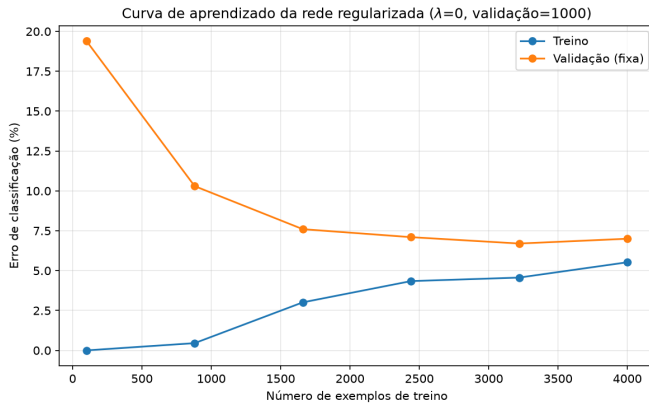


Figura 2: Curva de aprendizado para $\lambda = 0$, usando conjunto fixo de validação com 1000 exemplos.

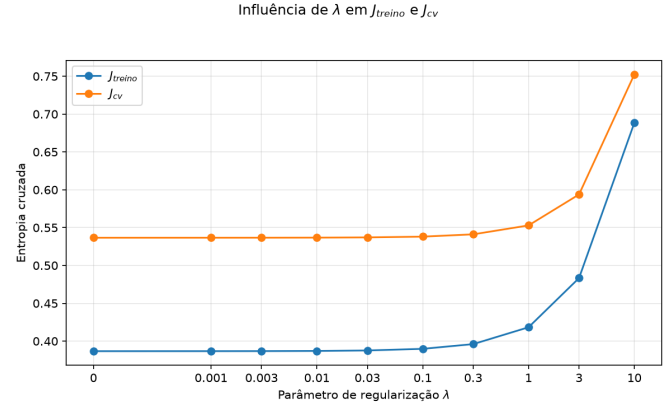


Figura 3: Entropia cruzada nos conjuntos de treinamento e validação em função de λ .

pouco convencionais, tornando plausível a associação visual com outra classe.

Essa inspeção qualitativa complementa a acurácia agregada: enquanto a métrica global informa a proporção total de acertos, os exemplos individuais revelam que parte dos erros está associada à ambiguidade da escrita manuscrita. Entretanto, como a figura mostra apenas os primeiros 25 erros, ela não permite afirmar quais confusões são mais frequentes no conjunto inteiro. Para uma análise por classe, seria necessário incluir uma matriz de confusão e métricas de precisão, revocação e F_1 por classe.

E. Similaridade entre classes e estrutura espacial

Para investigar a proximidade entre imagens de classes distintas, foi calculada a similaridade de cosseno entre os vetores

de pixels achatados. A Figura 7, apresentada no Apêndice, mostra os quatro pares interclasse de maior similaridade encontrados.

Os pares exibidos correspondem às classes 7 e 9 (0,9367), 3 e 8 (0,9309), 10 e 1 (0,9232), e 1 e 7 (0,9205). Portanto, exemplos com rótulos distintos podem apresentar vetores de pixels altamente alinhados no espaço de entrada. Em particular, vários desses pares compartilham traços curvos, diagonais ou regiões de alta intensidade semelhantes.

A vetorização de uma imagem 20×20 em um vetor de 400 atributos não elimina os valores dos pixels nem sua posição no vetor. Contudo, uma rede totalmente conectada não incorpora explicitamente relações locais entre pixels vizinhos, compartilhamento de pesos ou invariância a pequenas translações. Essas relações precisam ser aprendidas a partir dos dados.

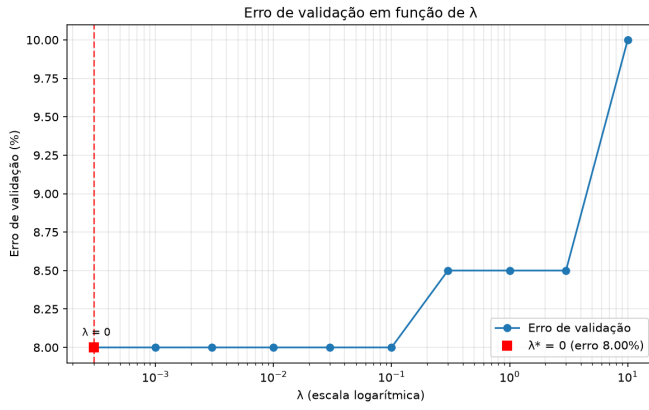


Figura 4: Erro de validação em função de λ , em escala logarítmica.

Redes neurais convolucionais introduzem esse viés indutivo por meio de filtros locais e pesos compartilhados, explorando mais diretamente padrões como bordas, curvas e continuidade de traços [1], [8]. Dessa forma, uma CNN constitui uma extensão natural para trabalhos futuros. O experimento de similaridade é apenas descritivo e não representa uma comparação empírica entre a MLP implementada e uma CNN.

F. Representações aprendidas na camada oculta

A Figura 8, apresentada no Apêndice, mostra os pesos associados aos 25 neurônios da camada oculta após sua reorganização em matrizes 20×20 . A visualização permite observar regiões com pesos positivos e negativos associadas a diferentes áreas da imagem de entrada.

Os neurônios não representam, em geral, dígitos completos e facilmente identificáveis. Em vez disso, seus pesos formam padrões distribuídos, que podem responder a combinações de traços, inclinações, regiões claras ou contornos. A classificação final resulta da combinação das ativações produzidas pelos vários neurônios ocultos, e não da atuação isolada de um único neurônio.

Esse comportamento é consistente com a função das camadas ocultas em redes multicamadas treinadas por retropropagação: construir representações intermediárias úteis para a decisão final [4], [5]. O GIF disponibilizado no repositório complementa essa análise ao mostrar a evolução temporal desses pesos durante o treinamento.

G. Síntese dos resultados

Os resultados mostram que a implementação reproduziu comportamentos esperados de uma rede neural totalmente conectada aplicada à classificação de dígitos manuscritos. O aumento da quantidade de dados reduziu a diferença entre os erros de treinamento e validação, evidenciando melhora de generalização. A regularização não trouxe ganho mensurável na faixa avaliada, e valores elevados de λ aumentaram o custo e o erro de validação.

Na comparação de métodos, o gradiente conjugado atingiu acurácias de validação elevadas com menos iterações

que o gradiente descendente. As configurações selecionadas alcançaram 93,1% e 93,8% de acurácia no teste para gradiente descendente e gradiente conjugado, respectivamente.

Por fim, a análise qualitativa dos erros e dos pares interclasse mostrou que parte das dificuldades está associada a escritas ambíguas e a padrões visuais compartilhados por classes distintas. Como limitações, permanecem a ausência de avaliação por classe, de matriz de confusão e de repetição dos experimentos com múltiplas sementes. Esses itens constituem extensões diretas para tornar a análise quantitativa mais robusta.

IV. CONCLUSÃO

A implementação manual de uma Rede Neural Artificial para classificação de dígitos manuscritos permitiu explorar como escolhas de arquitetura, de dados e de otimização se traduzem em desempenho, reproduzindo o comportamento esperado de uma rede com uma camada oculta: 93,1% de acurácia no conjunto de teste com gradiente descendente e 93,8% com gradiente conjugado.

Os experimentos resultaram em três resultados principais. Primeiro, a quantidade de dados mostrou-se determinante para a generalização: com 100 exemplos, o erro de validação chegou a 19,4%, enquanto com 3220 exemplos caiu para 6,7%, evidenciando que o sobreajuste em conjuntos pequenos é atenuado com mais dados. Segundo, a regularização não trouxe ganho mensurável de generalização, e valores elevados de λ degradaram o desempenho, indicando que, nesta arquitetura e volume de dados, o controle da magnitude dos pesos não era a limitação relevante. Terceiro, o gradiente conjugado convergiu muito mais rápido que o gradiente descendente, atingindo acurácias superiores com menos iterações, embora a diferença final no conjunto de teste tenha sido de apenas 0,7%.

A análise qualitativa complementou essas métricas agregadas. A inspeção dos erros mostrou que parte das falhas ocorre em dígitos com traços inclinados ou laços incompletos, e a similaridade de cosseno revelou que classes distintas podem ter vetores de pixels altamente alinhados, como nos pares 7–9 e 3–8. A visualização dos pesos da camada oculta mostrou que os neurônios não aprendem dígitos completos, mas padrões distribuídos que respondem a traços e contornos.

Como limitações, destaca-se que os resultados foram obtidos com uma única partição de dados e uma única *seed*, de modo que as comparações entre otimizadores devem ser lidas como evidência indicativa.

V. DISPONIBILIDADE DE CÓDIGO

O código-fonte desenvolvido para este trabalho, incluindo a implementação da rede neural, os experimentos realizados e os scripts utilizados para geração das figuras, está disponível publicamente no repositório: <https://github.com/luizlopes65/ProjetosMS571>. O repositório contém os arquivos necessários para reprodução dos principais experimentos apresentados neste trabalho.

REFERÊNCIAS

- [1] I. Goodfellow, Y. Bengio e A. Courville, *Deep Learning*. MIT Press, 2016, ISBN: 9780262035613. endereço: <https://www.deeplearningbook.org/>
- [2] W. S. McCulloch e W. Pitts, “A Logical Calculus of the Ideas Immanent in Nervous Activity”, *The Bulletin of Mathematical Biophysics*, v. 5, n. 4, pp. 115–133, 1943. DOI: 10.1007/BF02478259
- [3] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”, *Psychological Review*, v. 65, n. 6, pp. 386–408, 1958. DOI: 10.1037/h0042519
- [4] D. E. Rumelhart, G. E. Hinton e R. J. Williams, “Learning Representations by Back-Propagating Errors”, *Nature*, v. 323, n. 6088, pp. 533–536, 1986. DOI: 10.1038/323533a0
- [5] Y. LeCun, Y. Bengio e G. Hinton, “Deep Learning”, *Nature*, v. 521, n. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539
- [6] Y. LeCun, C. Cortes e C. J. C. Burges, *The MNIST Database of Handwritten Digits*, Acesso em: 21 set. 2026, 1998. endereço: <https://yann.lecun.com/exdb/mnist/>
- [7] A. Krogh e J. A. Hertz, “A Simple Weight Decay Can Improve Generalization”, em *Advances in Neural Information Processing Systems 4*, Morgan Kaufmann, 1991, pp. 950–957.
- [8] Y. LeCun, L. Bottou, Y. Bengio e P. Haffner, “Gradient-Based Learning Applied to Document Recognition”, *Proceedings of the IEEE*, v. 86, n. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791
- [9] A. Krizhevsky e G. Hinton, “Learning Multiple Layers of Features from Tiny Images”, University of Toronto, rel. técn., 2009.
- [10] A. Krizhevsky, I. Sutskever e G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks”, em *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.

VI. APÊNDICE

A. Comparação dos métodos de otimização

A Figura 5 apresenta a comparação entre gradiente descendente e gradiente conjugado para diferentes números de iterações e valores do parâmetro de regularização λ .

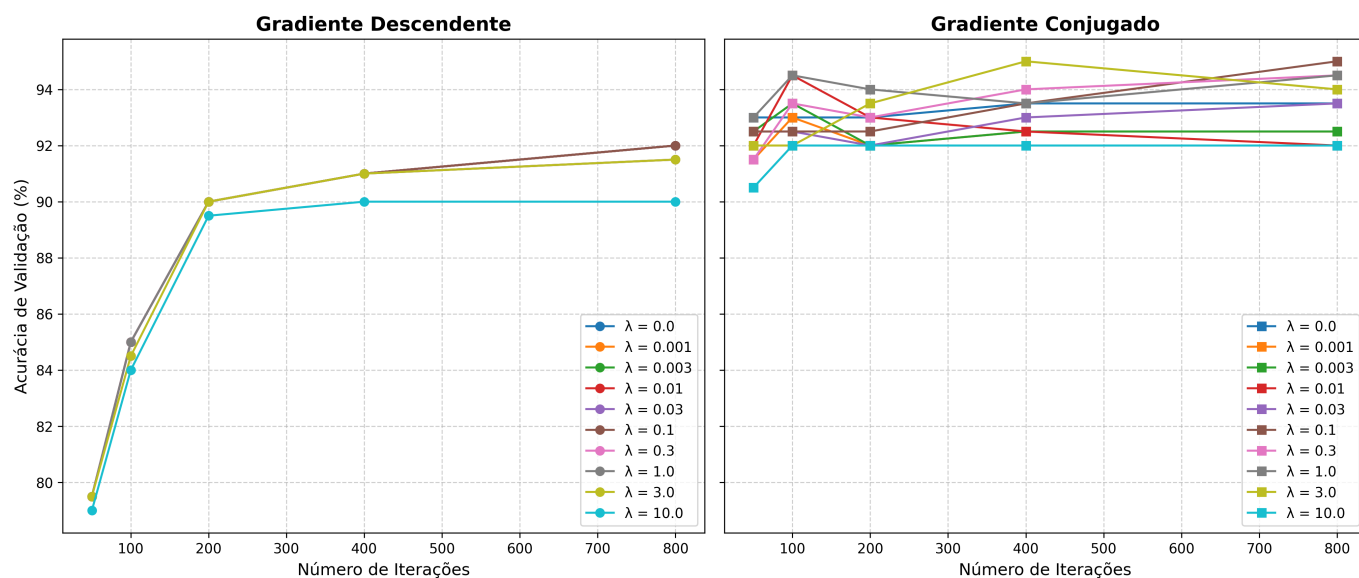


Figura 5: Comparação entre gradiente descendente e gradiente conjugado para diferentes valores de λ e números de iterações.

B. Exemplos de classificações incorretas

A Figura 6 apresenta os primeiros 25 exemplos classificados incorretamente pela configuração baseada em gradiente descendente com $\lambda = 0$. A figura complementa qualitativamente a análise apresentada na seção de resultados.

Erros | $\lambda=0$ | acurácia=93.10%

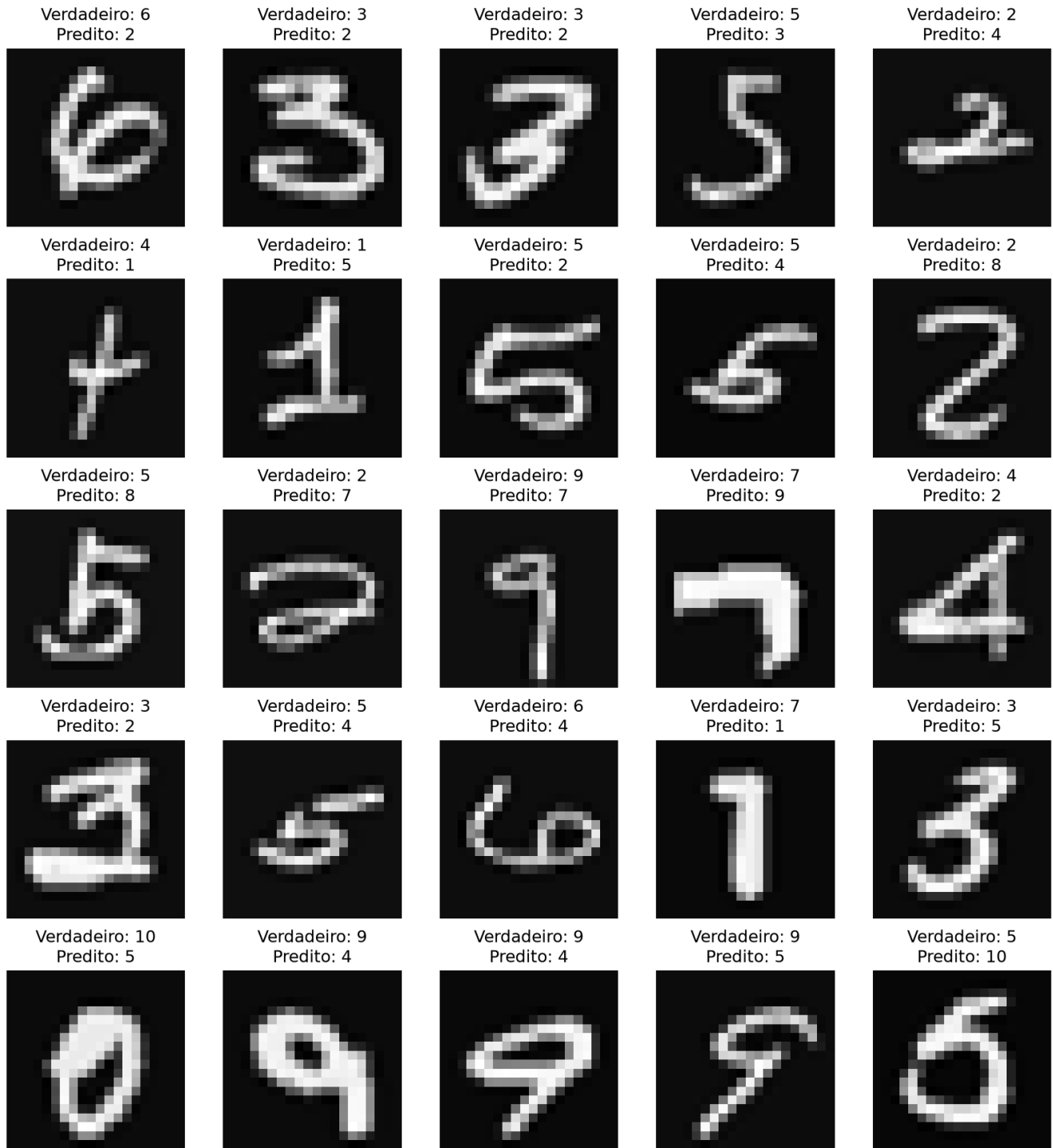


Figura 6: Amostra de classificações incorretas. Cada exemplo apresenta a classe verdadeira e a classe predita.

C. Pares de classes com alta similaridade de cosseno

A Figura 7 apresenta os quatro pares de imagens pertencentes a classes distintas com maior similaridade de cosseno encontrada no espaço vetorial de entrada.

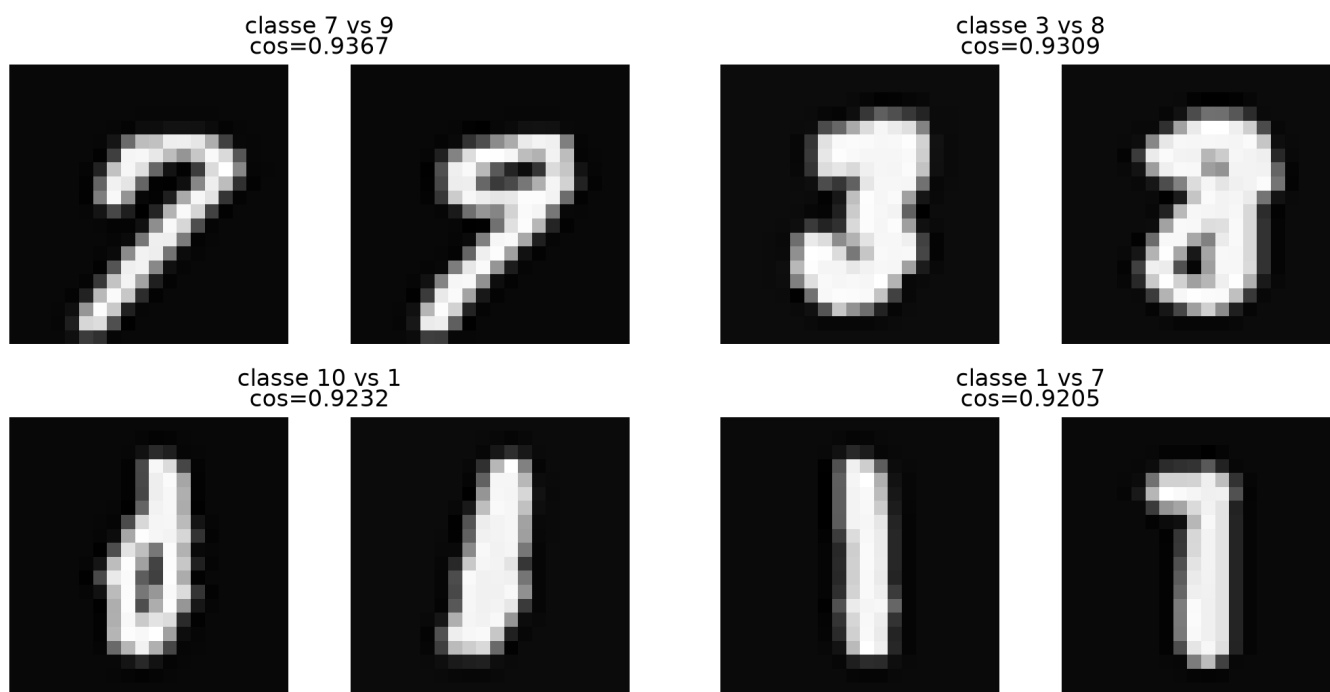


Figura 7: Quatro pares interclasse de maior similaridade de cosseno. As classes e os valores de similaridade são indicados acima de cada par.

D. Visualização dos pesos da camada oculta

A Figura 8 apresenta os pesos associados aos 25 neurônios da camada oculta após o treinamento, reorganizados em matrizes 20×20 para permitir sua inspeção visual.

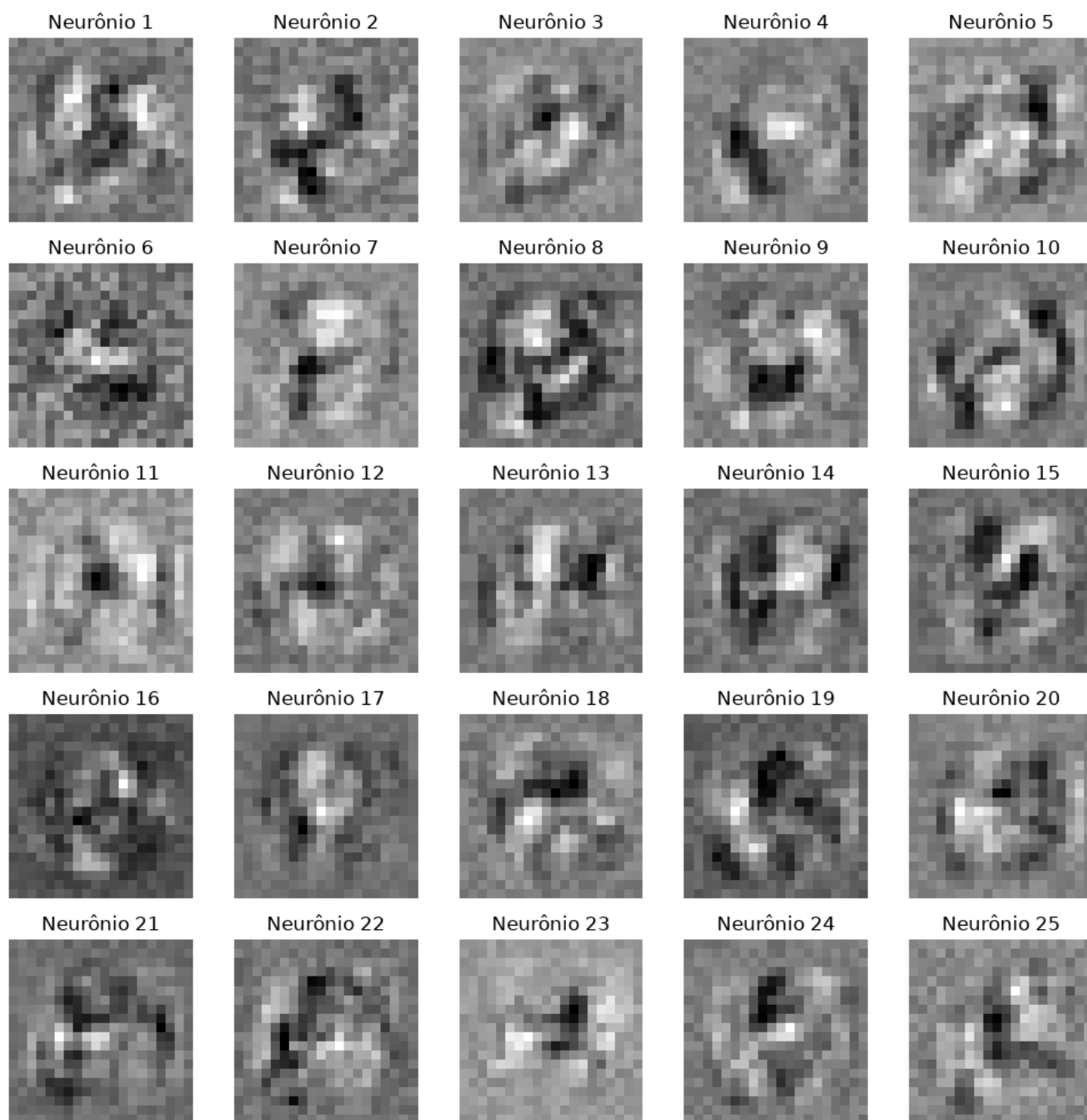


Figura 8: Visualização dos pesos associados aos neurônios da camada oculta após o treinamento.